

CPE484 HW 05: RTOS

CPE484: Real-Time Systems Development | Austin Driggs | 2026-03-05

For the programming assignment, submit you C code and screen snapshot. For the questions, provide your short answers.

Part 1: Simple Task to Calculate Pi

Instructions

- Add a task to the FreeRTOS main_blinky.c program
- Have this task calculate Pi
- Leave the other tasks also running - you should have 3 tasks
- Have the task run every 1 second
- Show source code and output from the program
- Did you encounter any surprises?

C Code

Editing FreeRTOS/Demo/Posix_GCC/main_blinky.c

Above the main_blinky function, I added:

```
/*
 * The tasks as described in the comments at the top of this file.
 */
static void prvCalculatePI_1( void * pvParameters );
```

This notifies the compiler that prvCalculatePI_1 exists before it is actually called in the code. It prevents "implicit declaration" errors by defining the function's signature at the top of the file.

Inside the main_blinky function, I added:

```
printf("creating PI 1 task\n");
xTaskCreate( prvCalculatePI_1,           /* The function that implements the task. */
            "Pi_1",                     /* The text name assigned to the task. */
            configMINIMAL_STACK_SIZE,  /* The size of the stack to allocate to the task. */
            NULL,                       /* The parameter passed to the task - not used. */
            mainQUEUE_SEND_TASK_PRIORITY, /* The priority assigned to the task. */
            NULL );                    /* The task handle is not used. */
```

This registers the Pi calculation with the FreeRTOS scheduler, allocating it a name, stack size, and priority. Without this specific call, the task would never be loaded into memory or granted CPU time to run.

Below the main_blinky function, I added:

```
static void prvCalculatePI_1( void * pvParameters ) {
    /* Prevent the compiler warning about the unused parameter. */
    (void) pvParameters;

    /* 500ms Delay */
    const TickType_t xDelay = 500 / portTICK_PERIOD_MS;

    double pi_4 = 0.0;
    // double pi_to_q = 0.0;
    int Flag = 1;
```

```

int i = 1;

for(;;) {
    if (Flag) {
        pi_4 += 1.0 / (double) i;
    }
    else {
        pi_4 -= 1.0 / (double) i;
    }

    printf( "PI = %.12f, i=%d\n", pi_4 * 4, i );

    i += 2;
    Flag = !Flag;

    vTaskDelay(xDelay);
}
}

```

This contains the infinite loop that performs the actual formula and toggles the addition/subtraction flag. The `vTaskDelay` tells the kernel to yield the CPU for 1000ms so the other two demo tasks can also execute.

I also learned that `i` needs incremented by 2 because its performing the [Leibniz formula for Pi](#):

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \dots = \sum_{i=0}^{\infty} \frac{(-1)^i}{2i+1}$$

Make the build for the demo

Ensure you are already in the FreeRTOS project repo and then run:

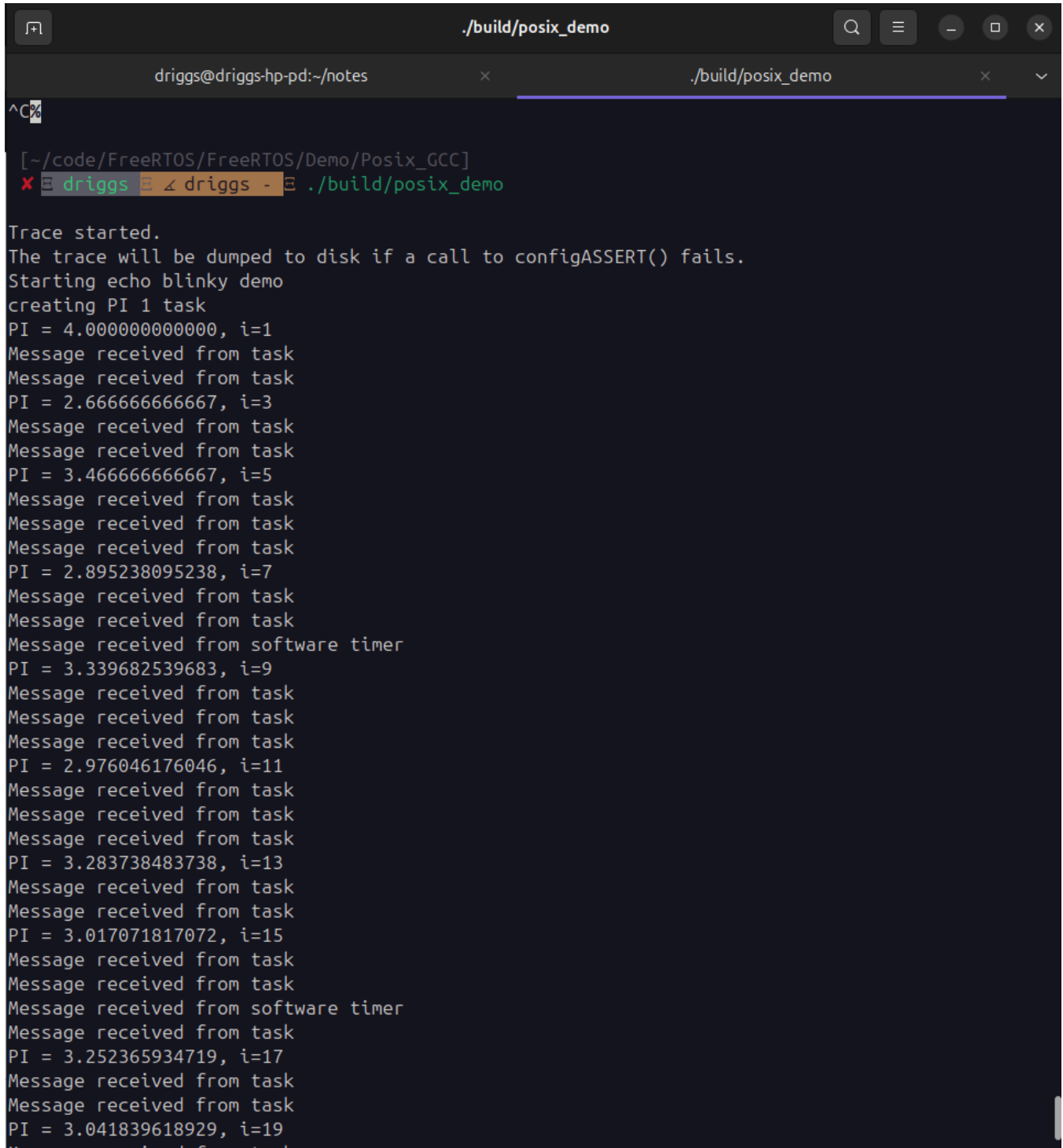
```

$ cd FreeRTOS/Demo/Posix_GCC/
$ make CFLAGS="-DUSER_DEMO=0"
$ ./build/posix_demo

```

Output of Task Running

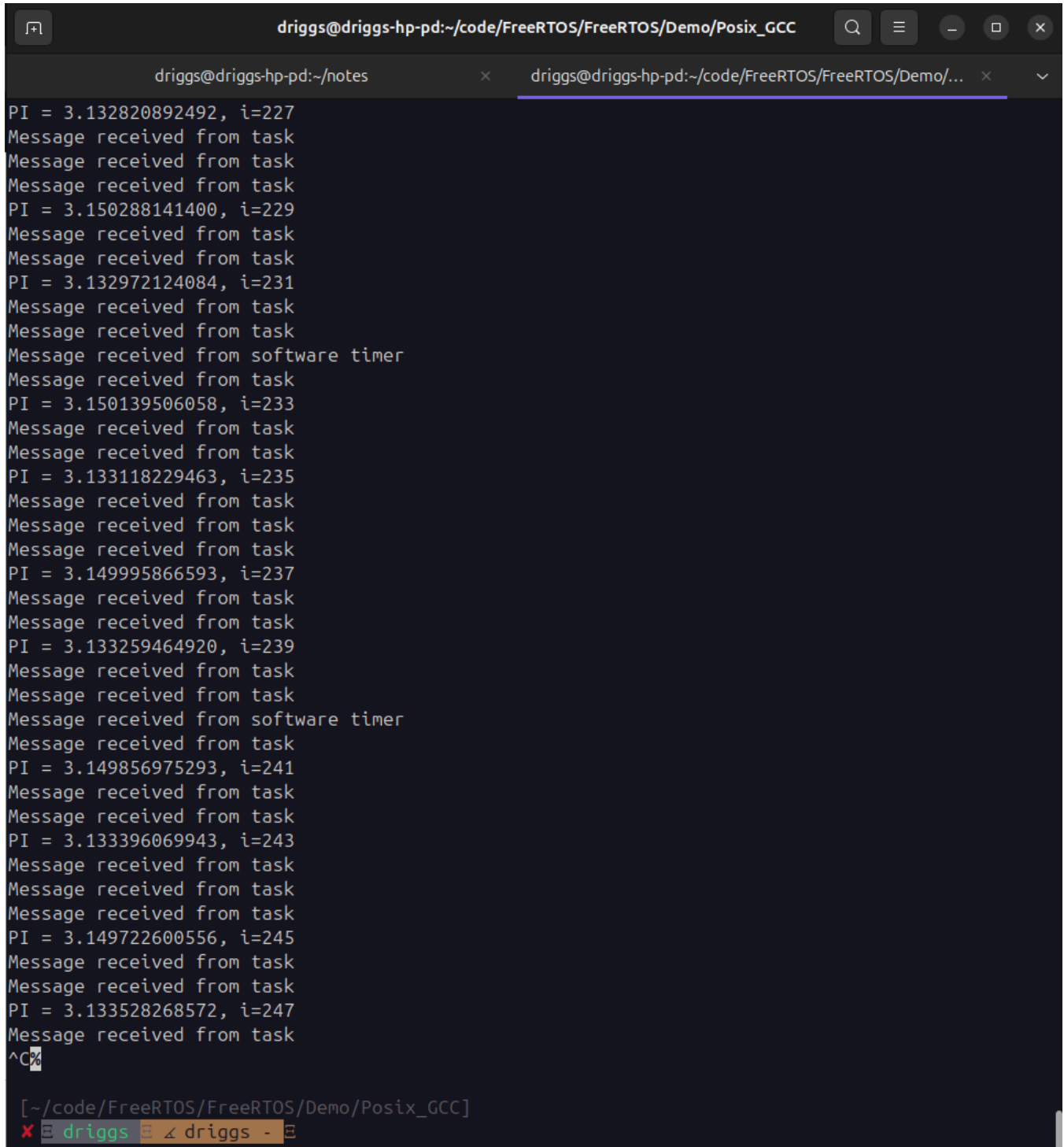
Running the code is shown below:



The screenshot shows a terminal window with the title bar `./build/posix_demo`. The terminal content is as follows:

```
driggs@driggs-hp-pd:~/notes x ./build/posix_demo  
^C%  
[~/code/FreeRTOS/FreeRTOS/Demo/Posix_GCC]  
x driggs driggs - ./build/posix_demo  
Trace started.  
The trace will be dumped to disk if a call to configASSERT() fails.  
Starting echo blinky demo  
creating PI 1 task  
PI = 4.000000000000, i=1  
Message received from task  
Message received from task  
PI = 2.666666666667, i=3  
Message received from task  
Message received from task  
PI = 3.466666666667, i=5  
Message received from task  
Message received from task  
Message received from task  
PI = 2.895238095238, i=7  
Message received from task  
Message received from task  
Message received from software timer  
PI = 3.339682539683, i=9  
Message received from task  
Message received from task  
Message received from task  
PI = 2.976046176046, i=11  
Message received from task  
Message received from task  
Message received from task  
PI = 3.283738483738, i=13  
Message received from task  
Message received from task  
PI = 3.017071817072, i=15  
Message received from task  
Message received from task  
Message received from software timer  
Message received from task  
PI = 3.252365934719, i=17  
Message received from task  
Message received from task  
PI = 3.041839618929, i=19  
Message received from task
```

I decided to stop the tasks as soon as I saw 3.14:



```
driggs@driggs-hp-pd:~/code/FreeRTOS/FreeRTOS/Demo/Posix_GCC
driggs@driggs-hp-pd:~/notes
PI = 3.132820892492, i=227
Message received from task
Message received from task
Message received from task
PI = 3.150288141400, i=229
Message received from task
Message received from task
PI = 3.132972124084, i=231
Message received from task
Message received from task
Message received from software timer
Message received from task
PI = 3.150139506058, i=233
Message received from task
Message received from task
PI = 3.133118229463, i=235
Message received from task
Message received from task
Message received from task
PI = 3.149995866593, i=237
Message received from task
Message received from task
PI = 3.133259464920, i=239
Message received from task
Message received from task
Message received from software timer
Message received from task
PI = 3.149856975293, i=241
Message received from task
Message received from task
PI = 3.133396069943, i=243
Message received from task
Message received from task
Message received from task
PI = 3.149722600556, i=245
Message received from task
Message received from task
PI = 3.133528268572, i=247
Message received from task
^C%
```

Did you encounter any surprises?

The biggest thing that surprised me is just how slowly the formula/task converged to even 3.14. For my code, it took until $i=237$, or about 119 loops, and that was just to get to 3.149996, which is still a very rough approximation of π depending on the task and how accurate you need to be.

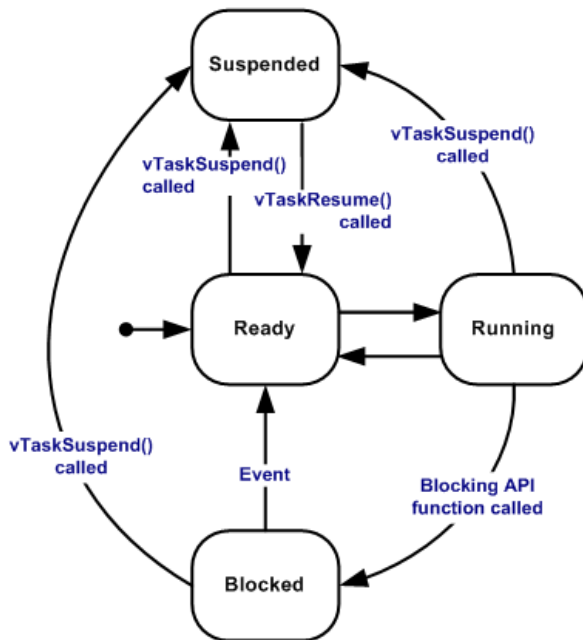
Part 2: Short Answer Questions

[5 Points] Use the “Mastering_the_FreeRTOS_Real_Time_Kernel-A_Hands_On_Tutorial.pdf” to answer the following questions:

- Read Section 3.6 (Pages 61 to 71) in the PDF.
- Review Example 4. Describe what happens when the `vTaskDelay()` function is being called?

When `vTaskDelay()` is called, it places the calling task into the Blocked state for a fixed number of tick interrupts. This blocked state allows the CPU to run other tasks.

[5 Points] Research the FreeRTOS Tasking state diagram. Describe in your own words each of the states: Suspended, Ready, Blocked, Running. Re-create the state diagram with transitions.



Source: <https://freertos.org/Documentation/02-Kernel/02-Kernel-features/01-Tasks-and-co-routines/02-Task-states>

- Running: Task is currently executing and using the CPU.
- Ready: Task is able to execute but waiting for its turn.
- Blocked: Task is waiting for either a temporal or external event.
- Suspended: Task is manually paused and ignored by the scheduler.

[5 Points] For your PI task you have running in FreeRTOS, describe in your own words what is happening with your task with respect to FreeRTOS states? Walk your PI task through the state diagram. What states is your PI task passing through?

- Ready to Running: Because it shares a priority level with the TX task, it waits in the Ready state until the scheduler selects it to return to the Running state for the next calculation.
- Running to Blocked: After calculated and printing the Pi value, the task calls `vTaskDelay()`, which moves it from the Running state to the Blocked state to wait for the next second.
- Blocked to Ready: Once the 1000ms timer expires, the kernel automatically moves the task from the Blocked state back to the Ready state.

[5 Points] For your PI task, is it a Simple or Complex task? Explain.

The Pi task is a Simple task. It follows the standard "continuous loop" structure of a basic FreeRTOS task, where it performs a single calculation, prints the result, and then enters the Blocked state to yield the processor. It does not interact with other tasks or handle any hardware interrupts.